

Digital Systems Project Assignment 2

Digital Watch: Stopwatch and Timer

Professor Jonathan H. Manton

This is the second of two assignments. Building on your settable watch from Assignment 1, you will add a stopwatch, a timer, and a display brightness control. You will also be introduced to formal verification and see first-hand how a synthesiser translates your code into actual hardware. Greater emphasis is placed on explaining the design decisions along the way.

Contents

1. Policies and Assessment	2
2. Automated Tests	2
3. Formal Verification	2
3.1. Up-Down Counter with Reset	2
3.2. Assertion File	4
3.3. Assertions, Valid States and Verification Checks	6
3.4. Configuration File	8
3.5. Expected Output	9
3.6. Examining Counterexamples	12
4. Stopwatch	15
4.1. Specification	15
4.2. Design Approach	16
4.3. Cascade Counter	16
4.4. Stopwatch Counter	17
4.5. Snapshot Multiplexer	18
4.6. Stopwatch Control	19
4.7. Stopwatch Integration	20
4.8. Programming the Board	20
5. Timer	20
5.1. Specification	21
5.2. Design Approach	21
5.3. Editable Countdown with Borrow Out	22
5.4. Timer Integration	22
5.5. Programming the Board	24
6. Brightness Control	24
6.1. Grey Code	24

6.2. Brightness Wrapper	25
7. Integration	25
7.1. Timepiece	26
7.2. Brightness Wrapper	27
7.3. Programming the Board	28
8. Synthesis	29
9. Enhancements	31
A. Common Pitfalls	32
B. Formal Verification Environment	32
C. Version History	33
C.1. Changes in Version 1.2	33
C.2. Changes in Version 1.1	33

1. Policies and Assessment

These are the same as for Assignment 1.

Back up your work regularly.

2. Automated Tests

The automated test suite, accessed by running `pytest` in the terminal, includes per-module functional tests, the formal verification tests introduced later in this assignment, and checks that no module has linting or compilation warnings or errors.

Run `pytest` now to confirm that all your files from Assignment 1 are in order before proceeding.

For the modules written as part of this assignment, some will have formal verification tests only, some will have cocotb tests only, and some will have both. Formal verification is not a panacea and is generally not the right tool to use for testing responses to sequences of inputs.

3. Formal Verification

Formal verification is introduced here using an up-down counter with reset as the running example.

3.1. Up-Down Counter with Reset

The up-down counter from Assignment 1 lacked reset capability.

Report 1

When a module needs to be extended mid-design, several approaches are possible: modify the original in place; copy it, modify the copy, and leave the original untouched; or copy it, modify the copy, and rewrite the original as a thin wrapper around the new module. Can you think of any other options? For this specific case, which approach would you take, and how would you name the resulting module or modules? Justify your choices and discuss their trade-offs relative to the alternatives.

SystemVerilog's support for default port values makes modifying the original module particularly attractive here. Adding `input logic rst = 1'b0` to the port list means existing instantiations that do not connect `rst` continue to work unchanged, while new instantiations can drive it explicitly. Any regression introduced during the modification would be caught by the existing test suite; the suite should then be extended to cover the new reset behaviour.

In practice, however, Verilator and Icarus Verilog both lack support for default port values; we therefore leave the existing module untouched and create a new one from a copy.

Task Implement the following module in `rtl/up_down_counter_rst.sv`, following all conventions from Assignment 1, including **elimination of all Verilator linting warnings**.

```
1 module up_down_counter_rst #(
2     parameter int MAX    = 2,
3     parameter int WIDTH = 2
4 ) (
5     input logic clk,
6     input logic rst,
7     input logic enable,
8     input logic up,
9     output logic [WIDTH-1:0] count
10 );
```

The `count` is initialised to 0.

When `rst` is high, `count` is set to 0 on the next rising clock edge, regardless of `enable` or `up`.

When `rst` is low and `enable` is low, `count` does not change.

When `rst` is low and `enable` is high, `count` increments if `up` is high and decrements if `up` is low, wrapping from `MAX` to 0 on increment and from 0 to `MAX` on decrement.

Unlike in Assignment 1, no predefined test benches are provided; instead, this module is verified formally. Formal verification is far more powerful than a test bench: rather than checking a finite set of inputs, it proves correctness for all possible inputs. Once you are happy with your implementation, the following command should report success:

```
sby -f sby/up_down_counter_rst.sby
```

The `-f` flag is essential when running `sby` more than once: it instructs `sby` to delete its output directory before starting. Without it, `sby` will refuse to run if the output directory already exists.

To understand the output, we first explain the assertion file and the configuration file; what to do when verification fails is covered in Section 3.6.

Pytest Run all tests, including formal verification, with a plain `pytest`. To run formal tests in isolation, use `pytest -m formal`; add `-v` to list each test, and `-s` to show `sby` output (which is suppressed when all checks pass).

3.2. Assertion File

The assertion file wraps the design under verification in a module that instantiates it and adds the formal properties to be checked. Because SVA concurrent assertions (which use the `property` and `sequence` keywords to express temporal properties) require the commercial Verific front-end and are therefore not available in open-source Yosys flows, we use immediate assertions instead: `assert` statements placed directly inside procedural blocks.

```
1 // -----
2 // Formal Verification
3 // -----
4 `timescale 1ns / 1ps
5
6 module up_down_counter_rst_assert #(
7     parameter int MAX    = 2,
8     parameter int WIDTH = 2
9 ) (
10     input logic clk,
11     input logic enable,
12     input logic rst,    // Takes priority over enable
13     input logic up,
14     output logic [WIDTH-1:0] count
15 );
16
17     up_down_counter_rst #(
18         .MAX (MAX),
19         .WIDTH(WIDTH)
20     ) u_counter (
21         .clk(clk),
22         .enable(enable),
23         .rst(rst),
24         .up(up),
25         .count(count)
26     );
27
28     // verilog_lint: waive-start always-comb
29
30     /* verilator lint_off WIDTHEXPAND */
31     /* verilator lint_off BLKSEQ */
32
33     // Ensure parameter values are sensible
34     initial assert (MAX >= 0);
35     initial assert ((1 << WIDTH) > MAX);
36
37     // Ensure count remains within range at all times
38     always @(*) assert (count <= MAX);
```

```

39
40 // Detect start-up
41 logic start = 1'b1;
42 always @(posedge clk) start <= 1'b0;
43
44 // Check initial value and reset value
45 always @(posedge clk) if (start || $past(rst)) a_reset : assert (count == 0);
46
47 // Check hold
48 always @(posedge clk)
49     if (!start && !$past(enable) && !$past(rst))
50         a_hold : assert (count == $past(count));
51
52 // Check count up
53 logic wrap_up;
54 always @(posedge clk) begin
55     wrap_up = ($past(count) == MAX && count == 0);
56     if (!start && $past(enable) && !$past(rst) && $past(up))
57         a_count_up : assert (count == $past(count) + 1 || wrap_up);
58 end
59
60 // Check count down
61 logic wrap_down;
62 always @(posedge clk) begin
63     wrap_down = ($past(count) == 0 && count == MAX);
64     if (!start && $past(enable) && !$past(rst) && !$past(up))
65         a_count_down : assert (count == $past(count) - 1 || wrap_down);
66 end
67
68 endmodule

```

Line 4 adds a timescale directive merely to suppress a Verilator warning.

Lines 6–15 declare a wrapper module with the same ports as `up_down_counter_rst`. The formal verification tools verify this wrapper, attempting to prove that no sequence of inputs can cause an assertion to fail; by instantiating `up_down_counter_rst` inside it (lines 17–26), we direct that proof toward the actual design. Had the open-source tools supported it, SystemVerilog’s `bind` mechanism would have made the wrapper unnecessary.

Line 28 is a Verible linter pragma: a directive embedded in a comment so that other tools see only a comment, while the linter treats it as an instruction. Without it, VS Code would warn about our `always @(*)` statements; the pragma suppresses the `always-comb` warning without touching the code itself.

Lines 30–31 are Verilator pragmas. The first disables the bit-width check; bit-width errors are unlikely in this context, and the warning would only add noise. The second disables the warning about blocking assignments in `always @(posedge clk)` blocks. Blocking assignments are used deliberately here so that a variable such as `wrap_up` is updated immediately and can be read by the assertion in the same block.

Lines 34–35 check at initialisation that the parameter values are sensible: if formal verification fails, we do not want to waste time diagnosing the failure only to discover the parameters were nonsensical. The second assertion neatly checks that `WIDTH` is wide enough to hold `MAX`.

Line 38 asserts that `count` never exceeds `MAX`.

Lines 41–42 define the flag `start`, which is high throughout cycle 0 and falls to zero just after the first rising edge via non-blocking assignment. It is used, among other things, to assert that `count` is zero at initialisation.

Line 45 checks two things: that `count` is zero at initialisation and zero after a reset. Because rising edges demarcate cycles, each `always @(posedge clk)` block can be understood either as occurring at the end of the previous cycle or at the start of the current cycle. By convention, it is said to occur at the start of the current cycle. The assertions within the block, however, check the state at the end of the previous cycle: signals settle just after a rising edge and remain constant until the next edge. The first time line 45 executes is therefore placed at cycle 1, but it checks conditions from cycle 0.

Within an `always @(posedge clk)` block attributed to cycle k , `rst` holds its value from cycle $(k - 1)$ and `$past(rst)` holds its value from cycle $(k - 2)$. In particular, at cycle 1, `$past(rst)` is undefined: there is no cycle (-1) . From cycle 2 onwards, `$past(rst)` is well defined.

The label `a_reset` on line 45 makes `sby`'s output easier to interpret: when an assertion fails, `sby` reports it by label if one is present, and by line number otherwise.

Lines 48–50 check that if neither `enable` nor `rst` was high, then `count` retains its previous value. Without `!start`, the assertion would fail at the first clock edge because `$past(count)` is undefined there, referring to cycle (-1) .

Lines 53–58 check that the counter increments correctly. If `enable` is high, `rst` is low, and `up` is high, `count` must either increment by one or wrap to zero if it was at `MAX`.

Lines 61–66 perform the symmetric check for counting downwards. If `enable` is high, `rst` is low, and `up` is low, `count` must either decrement by one or wrap to `MAX` if it was at zero.

When a design passes formal verification, it is worth reflecting on what has actually been proved. The proof establishes not that this module *is* an up-down counter with reset, but that it satisfies the properties captured by the assertions above. Whether those properties fully capture the concept of an up-down counter is a question about language, not logic; a lawyer might ask whether any reasonable person would consider the specification complete. Engineers can afford to be more pragmatic: provided nothing more is asked of this module than what the assertions guarantee, it is fit for purpose.

Report 2

Do the assertions above fully capture the behaviour of an up-down counter with reset? If not, identify any behaviours left unspecified, or any properties a reasonable person might expect that the assertions do not guarantee.

Report 3

The configuration file verifies the counter for three specific values of `MAX`. Since this does not cover all possible values, how would you formally prove correctness when this module is used in a larger design?

3.3. Assertions, Valid States and Verification Checks

The assertions are checked once per clock cycle. The formal verification tools assume ideal logic: signal values update just after a rising edge and remain constant until the next.

A state at cycle n is *valid* if no assertions fail at that cycle.

The two checks use this definition differently, as we now describe.

The formal verification tools run two checks: a bounded model check (BMC), which verifies that no assertion fails in the first k cycles, and a k -induction check, which verifies that any sequence of k consecutive valid states must be followed by another valid state. When both pass at the same depth k , together they constitute a complete proof of correctness for all time: the BMC establishes that the first k steps are assertion-free, and the induction check extends this guarantee indefinitely.

We use assertions in three block types: **initial**, **always @(*)**, and **always @(posedge clk)**. Each produces a counterexample when it fails; interpreting these correctly requires understanding how the two checks treat each block type. Since signals are constant within a cycle, it might seem that **always @(*)** and **always @(posedge clk)** are interchangeable. They are not.

An assertion in an **initial** block applies only to cycle 0. The induction checker ignores it, since induction requires assertions that can apply starting from any point in time, not just the initial state.

An assertion in an **always @(*)** block applies to all cycles, including cycle 0. Both checkers consider a state at cycle 0 to be invalid if it fails any such assertion, just as one would expect.

An assertion in an **always @(posedge clk)** block is attributed to the cycle being entered: the first time it triggers, it is attributed to cycle 1. Two things are natural: it is natural for the checker to attribute the assertion to the new cycle, and it is equally natural that the assertion checks the state of the previous cycle, as explained below. This mismatch does not compromise the validity of formal verification, but readers should keep it in mind when interpreting counterexample output.

Rising edges demarcate cycles: each edge marks the end of one cycle and the beginning of the next. The period before an edge is the *previous cycle*; the period after it is the *current cycle*. The first time **always @(posedge clk)** triggers, the circuit enters cycle 1. The checker adopts this labelling, which is entirely natural.

Because flip-flop outputs are assigned via non-blocking assignment, they take effect only after all procedural blocks have finished. Inside an **always @(posedge clk)** block, signal values are therefore those from just before the rising edge, that is, from the previous cycle. In particular, at cycle 1 the assertions check the state at cycle 0.

To make this concrete, suppose the induction checker initialises the state at cycle 0 to **start=1** and **count=3**. No assertions are violated because no rising edge has yet occurred. When the first rising edge arrives and line 45 executes, the prover records a violation at cycle 1, but the condition it checks concerns cycle 0. Someone reading “invalid at step 1” would naturally examine cycle 1, while the actual problem lies in cycle 0.

The k -induction check for our counter fails when $k = 1$, an artefact of the mismatched labelling. From the counter’s logic alone, if the state at cycle k is valid then the state at cycle $k + 1$ will also be valid, so one would expect the counter to pass 1-induction. The subtlety is that 1-induction is genuine 1-induction for **always @(*)** assertions, but behaves like 0-induction for **always @(posedge clk)** assertions, since those check the previous cycle’s state. The counter therefore requires 2-induction to pass.

To see why, consider the 1-induction counterexample. The checker places **start=1** and **count=3** at cycle 0. No **always @(*)** assertions fail, so cycle 0 is deemed valid. At the first rising edge, labelled cycle 1, line 45 executes and finds **count** non-zero at cycle 0, recording

a violation at cycle 1. A valid cycle 0 has thus led to a violation at cycle 1, so 1-induction fails, even though the counter is correct.

This artefact does not compromise the soundness of formal verification. The looser constraint on cycle 0 gives the induction checker more freedom to construct an adversarial initial state, making induction *harder* to pass, not easier. Any design that does pass is genuinely correct; the artefact merely forces a higher depth. For our counter, $k = 2$ suffices: the 2-induction hypothesis covers two consecutive valid states, which is enough to exclude the spurious counterexample above.

3.4. Configuration File

The configuration file `sby/up_down_counter_rst.sby` is shown below, followed by a discussion of each part keyed to the line numbers.

```

1  [options]
2  mode prove
3  depth 3
4
5  [engines]
6  smtbmc z3
7
8  [tasks]
9  max_4
10 max_15
11 max_60
12
13 [script]
14 read -sv -formal up_down_counter_rst.sv
15 read -sv -formal up_down_counter_rst_assert.sv
16
17 max_4:
18 chparam -set MAX 4 up_down_counter_rst_assert
19 chparam -set WIDTH 3 up_down_counter_rst_assert
20
21 max_15:
22 chparam -set MAX 15 up_down_counter_rst_assert
23 chparam -set WIDTH 4 up_down_counter_rst_assert
24
25 max_60:
26 chparam -set MAX 60 up_down_counter_rst_assert
27 chparam -set WIDTH 6 up_down_counter_rst_assert
28 --
29
30 prep -top up_down_counter_rst_assert
31
32 [files]
33 rtl/up_down_counter_rst.sv
34 sby/up_down_counter_rst_assert.sv

```

Lines 1–3 set the proof depth to 3, which governs both checkers. The base-case checker verifies that no assertion is violated in cycles 0, 1, or 2. The induction checker searches for a different kind of counterexample: does there exist a run of three consecutive cycles with all assertions satisfied, immediately followed by a cycle that violates one? If no such run exists, the property holds for all time.

Lines 5–6 select `z3` as the proof engine. Several SMT solvers are supported; `z3` is a natural choice, being general-purpose, actively maintained, widely used in both academia and industry, and well-suited to the bit-vector arithmetic that arises in hardware verification.

Lines 8–11 define the three task names: `max_4`, `max_15`, and `max_60`. These are chosen to be self-documenting, but `sby` treats them as opaque identifiers; the corresponding value of `MAX` is set explicitly for each task later in the file.

Lines 13–15 read the source files, applying two flags to each. Because `sby` copies the files listed in the `[files]` section into its working directory before running the script, only bare filenames appear here, without any path prefix. The file `up_down_counter_rst.sv` is the design under verification; the assertion wrapper `up_down_counter_rst_assert.sv` instantiates it and adds the formal properties. The flag `-sv` selects SystemVerilog parsing. The flag `-formal` defines the preprocessor macro `FORMAL`, which is useful when assert statements live in the same file as the module: guarding them with ``ifdef FORMAL ... `endif` makes them invisible to synthesis tools, which never define `FORMAL`.

Prefixing a script line with a task name, as on line 17, restricts it to that task alone. A different task takes over when its name appears, as on line 21, or the restriction is lifted entirely by `--`, as on line 28; from line 30 onward every task runs the same commands.

Lines 18–19, 22–23, and 26–27 use `chparam` to set `MAX` and `WIDTH` for each task.

Line 30 triggers the preparation phase: `sby` invokes `yosys` to elaborate the RTL, resolve parameters, and produce a netlist in the form expected by the formal backend. The `-top` flag identifies `up_down_counter_rst_assert` as the top-level module.

Lines 32–34 list the source files for `sby` to copy into its working directory. Paths are relative to the directory from which `sby` is invoked; the output directory for each task is created as a subdirectory alongside the `.sby` file.

3.5. Expected Output

Because `sby` cannot verify over all possible parameter values, we select a representative sample of `MAX` values and define each as a separate task in the configuration file `up_down_counter_rst.sby`: `max_4`, `max_15`, and `max_60`. When run without a task name, the command above launches all tasks in parallel, producing interleaved output; the third column of each line identifies which task produced it.

To run only `max_4`, append the task name:

```
sby -f sby/up_down_counter_rst.sby max_4
```

The first few lines of the output look like this:

```
root@8cb6c810b333:/workspace# sby -f sby/up_down_counter_rst.sby max_4
SBY 5:10:26 [sby/up_down_counter_rst_max_4] Removing directory '/workspace/
sby/up_down_counter_rst_max_4'.
SBY 5:10:26 [sby/up_down_counter_rst_max_4] Copy '/workspace/rtl/
up_down_counter_rst.sv' to '/workspace/sby/
up_down_counter_rst_max_4/src/up_down_counter_rst.sv'.
SBY 5:10:26 [sby/up_down_counter_rst_max_4] Copy '/workspace/sby/
up_down_counter_rst_assert.sv' to '/workspace/sby/
up_down_counter_rst_max_4/src/up_down_counter_rst_assert.sv'.
```

The first three columns (tool name, timestamp, and task identifier) are identical on every line. Dropping them, the output is presented below in four parts, with discussion interwoven.

```

Removing directory '/workspace/sby/up_down_counter_rst_max_4'.
Copy '/workspace/rtl/up_down_counter_rst.sv' to '/workspace/sby/
  up_down_counter_rst_max_4/src/up_down_counter_rst.sv'.
Copy '/workspace/sby/up_down_counter_rst_assert.sv' to '/workspace/sby/
  up_down_counter_rst_max_4/src/up_down_counter_rst_assert.sv'.
engine_0: smtbmc z3
base: starting process "cd sby/up_down_counter_rst_max_4/src; yosys -ql
  ../model/design.log ../model/design.js"
base: finished (returncode=0)
prep: starting process "cd sby/up_down_counter_rst_max_4/model; yosys -ql
  design_prep.log design_prep.js"
prep: finished (returncode=0)
smt2: starting process "cd sby/up_down_counter_rst_max_4/model; yosys -ql
  design_smt2.log design_smt2.js"
smt2: finished (returncode=0)
engine_0.basecase: starting process "cd sby/up_down_counter_rst_max_4;
  yosys-smtbmc -s z3 --presat --noprogess -t 3 --append 0 --dump-vcd
  engine_0/trace.vcd --dump-yw engine_0/trace.yw --dump-vlogtb
  engine_0/trace_tb.v --dump-smtc engine_0/trace.smtc model/design_smt2.smt2"
engine_0.induction: starting process "cd sby/up_down_counter_rst_max_4;
  yosys-smtbmc -s z3 --presat -i --noprogess -t 3 --append 0 --dump-vcd
  engine_0/trace_induct.vcd --dump-yw engine_0/trace_induct.yw --dump-vlogtb
  engine_0/trace_induct_tb.v --dump-smtc engine_0/trace_induct.smtc
  model/design_smt2.smt2"

```

The first block is mostly housekeeping: **sby** copies the relevant source files to its output directory, then invokes **yosys** in three sequential passes (**base**, **prep**, **smt2**) to compile the design into SMT-LIB format, a logical representation amenable to SMT solving. **Yosys** is best known as an open-source synthesis tool, but here it is used to translate hardware descriptions into a form amenable to formal analysis.

Two instances of **yosys-smtbmc** are then launched in parallel: one to check the base case and one to check the inductive step of a k -induction proof, as explained below. The solver is **z3**, a widely used SMT solver developed by Microsoft Research.

The configuration file specifies a depth of 3, which governs both checks. The base-case checker verifies that no assertion fails at clock cycles 0, 1, or 2, starting from the known initial state. The induction checker runs independently, without constraining the starting state to be the initial state: it asks whether there exist any three consecutive cycles in which all assertions hold, but the successor cycle violates one. If no such sequence exists, the inductive step holds, and together with the base case, k -induction guarantees the assertions hold for all time.

```

engine_0.basecase: ## 0:00:00 Solver: z3
engine_0.induction: ## 0:00:00 Solver: z3
engine_0.basecase: ## 0:00:00 Checking assumptions in step 0..
engine_0.induction: ## 0:00:00 Trying induction in step 3..
engine_0.basecase: ## 0:00:00 Checking assertions in step 0..
engine_0.basecase: ## 0:00:00 Checking assumptions in step 1..
engine_0.basecase: ## 0:00:00 Checking assertions in step 1..
engine_0.basecase: ## 0:00:00 Checking assumptions in step 2..
engine_0.basecase: ## 0:00:00 Checking assertions in step 2..
engine_0.basecase: ## 0:00:00 Status: passed
engine_0.basecase: finished (returncode=0)
engine_0.basecase: Status returned by engine for basecase: pass

```

The second block shows both solvers running in parallel. Focusing on the base-case checker: it verifies that no assertion fails at clock cycle 0, then cycle 1, then cycle 2. Having found no violation across all three cycles, and since the configured depth is 3, its job is done. The base case passes.

```
engine_0.induction: ## 0:00:00 Trying induction in step 2..
engine_0.induction: ## 0:00:00 Trying induction in step 1..
engine_0.induction: ## 0:00:00 Temporal induction successful.
engine_0.induction: ## 0:00:00 Status: passed
engine_0.induction: finished (returncode=0)
engine_0.induction: Status returned by engine for induction: pass
```

The induction checker's output was interleaved with the previous block but finishes here. Its goal is to prove that no assertion is ever violated: for all n , no assertion fails at cycle n . Unlike the base-case checker, it imposes no constraint on the starting state, so the proof must cover all possible mid-execution states, not just those reachable immediately from startup.

At **step 3**, the checker tries to prove that for all n , regardless of the state at cycle $n - 1$, no transition can lead to an assertion violation at cycle n . This is an overly strong condition, relying on the transition relation alone. Since the checker does not stop here, we know it found a counterexample; had the proof succeeded, it would have reported success immediately.

At **step 2**, the checker restricts attention to valid states at cycle $n - 1$ and tries to prove, for all n , that no valid state at cycle $n - 1$ leads to an invalid state at cycle n . The checker finds a counterexample: a valid state at cycle $n - 1$ from which the design steps into an invalid state at cycle n . This is possible because validity at cycle $n - 1$ is a weaker condition than one might expect, as was explained in Section 3.3.

At **step 1**, the checker restricts attention to valid states at cycle $n - 2$ and asks whether, for any n , such a state can transition through a valid state at cycle $n - 1$ and then into an assertion violation at cycle n . No such sequence exists: the proof succeeds, and **Temporal induction successful** is reported.

In summary, each step widens the window of required valid states before the potential violation, restricting the set of starting states the checker must consider. Once that set is narrow enough that no violating run survives, the inductive step is proved. Together with the base-case result, the k -induction proof is complete.

```
summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
summary: engine_0 (smtbmc z3) returned pass for basecase
summary: engine_0 (smtbmc z3) returned pass for induction
summary: engine_0 did not produce any traces
summary: successful proof by k-induction.
DONE (PASS, rc=0)
```

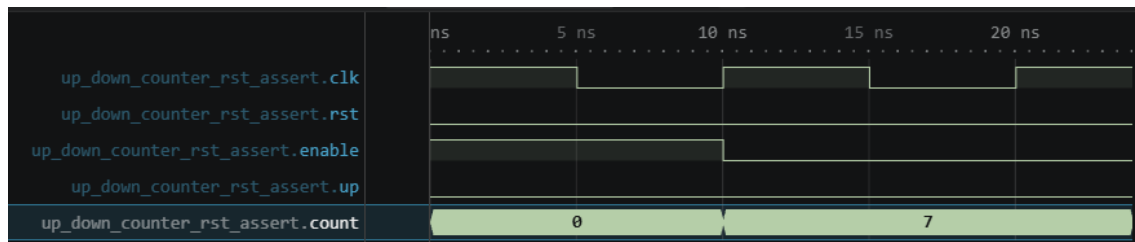
The final block confirms that the proof by k -induction succeeded: the module satisfies its assertions under all possible inputs and for all time. The line reporting no traces is only meaningful when verification *fails*: as will be discussed in Section 3.6, a failing assertion causes **sby** to record a waveform of the precise input sequence that leads to the violation. Here no counterexample was found, so no traces were generated.

3.6. Examining Counterexamples

To see what a failing verification looks like, consider an up-down counter in which the count-down wrap is omitted: rather than wrapping to MAX when count reaches zero, the code subtracts one unconditionally and lets the bit-vector underflow. Excerpts of the `sby` output for the `max_4` task are shown below in three parts, with prefixes and irrelevant lines removed.

```
## 0:00:00 BMC failed!
## 0:00:00 Assert failed in up_down_counter_rst_assert:
      up_down_counter_rst_assert.sv:38.14-38.36 (
        $assert$up_down_counter_rst_assert.sv:38$294)
## 0:00:00 Writing trace to VCD file: engine_0/trace.vcd
## 0:00:00 Writing trace to Verilog test bench: engine_0/trace_tb.v
## 0:00:00 Writing trace to constraints file: engine_0/trace.smtc
## 0:00:00 Writing trace to Yosys witness file: engine_0/trace.yw
```

The output indicates that the base-case checker found a violation: the unlabelled assertion on line 38 was falsified. A counterexample was saved to four files in `sby/up_down_counter_rst_max_4/engine_0`; the relevant one is `trace.vcd`, which can be opened in VS Code to bring up the VaporView waveform viewer (as in Assignment 1). Selecting the relevant signals gives the trace shown below.



The failed assertion is the range check on line 38 (`count <= MAX`). The trace is the counterexample `sby` found: the inputs `enable = 1`, `rst = 0`, and `up = 0` are applied while `count = 0`, and since the wrap is missing, count underflows to $2^{\text{WIDTH}} - 1$, which exceeds MAX.

```
## 0:00:00 Trying induction in step 2..
## 0:00:00 Trying induction in step 1..
## 0:00:00 Trying induction in step 0..
## 0:00:00 Temporal induction failed!
## 0:00:00 Assert failed in up_down_counter_rst_assert:
      up_down_counter_rst_assert.sv:38.14-38.36 (
        $assert$up_down_counter_rst_assert.sv:38$294)
## 0:00:00 Writing trace to VCD file: engine_0/trace_induct.vcd
## 0:00:00 Writing trace to Verilog test bench: engine_0/
      trace_induct_tb.v
## 0:00:00 Writing trace to constraints file: engine_0/trace_induct.
      smtc
## 0:00:00 Writing trace to Yosys witness file: engine_0/
      trace_induct.yw
```

The induction checker fails for the same reason. It searches for an arbitrary starting state and a sequence of inputs such that no assertion is violated in the first three cycles, but one is violated on the fourth; count can begin at any value the solver finds convenient.

Its counterexample is saved to `trace_induct.vcd`: opening it in VaporView shows `count` taking the values 3, 4, 0, then 7. Starting from `count = 3`, three up-count steps give 3, 4, 0, all within range (the wrap is correctly implemented); a final down-count step then causes `count` to underflow to 7, violating the range assertion.

```
##    0:00:00  Status: failed
finished (returncode=1)
Status returned by engine for basecase: FAIL
terminating process
Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
engine_0 (smtbmc z3) returned FAIL for basecase
counterexample trace [basecase]: sby/up_down_counter_rst_max_4/engine_0
/trace.vcd
failed assertion up_down_counter_rst_assert.
  $assert$up_down_counter_rst_assert.sv:38$294 at
  up_down_counter_rst_assert.sv:38.14-38.36 step 1
counterexample trace [induction]: sby/up_down_counter_rst_max_4/
engine_0/trace_induct.vcd
failed assertion up_down_counter_rst_assert.
  $assert$up_down_counter_rst_assert.sv:38$294 at
  up_down_counter_rst_assert.sv:38.14-38.36
DONE (FAIL, rc=2)
The following tasks failed: ['max_4']
```

The final summary confirms that both the base-case and induction checks failed, identifies the paths to the two counterexample trace files, and reports the specific assertion that was violated in each case, including its source location (`up_down_counter_rst_assert.sv`, line 38). The overall result is **FAIL** with return code 2, and `sby` lists the tasks that failed. The return code is useful when `sby` is run as part of an automated flow: a non-zero exit code signals failure to the calling script or build system.

Finding Counterexample Waveforms. Whenever `sby` reports a failure, **you will need to open the counterexample trace files** in VaporView. The paths to these files are printed in `sby`'s output, on lines labelled `counterexample trace`. For this run they are `sby/up_down_counter_rst_max_4/engine_0/trace.vcd` (base case) and `sby/up_down_counter_rst_max_4/engine_0/trace_induct.vcd` (induction). In future runs the filenames will differ, so read them directly from the output.

Valid Versus Reachable. The discussion below of k -induction's limitations hinges on the distinction between these two notions. The following example makes it concrete.

```
1 always_comb
2   if (count < 10) next_count = (count == 9) ? 0 : count + 1;
3   else next_count = (count == 19) ? 10 : count + 1;
```

This defines two independent counters: one cycling through 0–9 and the other through 10–19. No sequence of inputs can move the system from one range to the other. If `count` is initialised to 0, only the states 0–9 are reachable; the states 10–19 are not. Formally, a state is *reachable* if the design can arrive at it from the initial state defined by Verilog `initial` statements. A state is *valid* if it violates no assertion (Section 3.3). A *valid sequence* is a sequence of valid states in which each consecutive pair is connected by a transition that the design permits under some input. Not every valid state need be reachable.

Limitations of k -induction. Because the induction checker starts from an arbitrary state, its counterexamples are not guaranteed to be reachable. A *spurious* counterexample is one whose starting state is valid but unreachable from the true initial state of the design. When a spurious counterexample is present, the induction step fails even if the design is correct, and the proof cannot succeed until it is ruled out.

The simplest approach is to try increasing the depth: a longer lead-in of valid states is harder to construct, so some spurious counterexamples are ruled out as the depth grows.

However, this does not always work, particularly for designs with an `enable` input. A spurious counterexample found at one depth can be adapted to any greater depth by prepending additional cycles with `enable` held low (since nothing changes while `enable` is low, each such cycle remains valid). This exact problem arises in our counter if the following assertion is removed:

```
1 always @(*) assert (count <= MAX);
```

Without it, the induction checker cannot succeed at any depth. It can initialise `count` to a value greater than `MAX`, then hold `enable` low indefinitely without violating any assertion. Setting `enable` and up high then causes `count` to wrap to zero, but because it wraps from a value other than `MAX`, the transition violates the count-up assertion.

Task Temporarily comment out the `assert (count <= MAX)` line (shortcut: `Ctrl` + `/`), run `sby`, and observe that the induction step now fails. The paths to the counterexample trace files are printed in `sby`'s output; open them in VaporView. **Restore the assertion file before continuing.**

Report 4

Include the counterexample waveform. The general reason for this failure was explained above; use the waveform to identify the specific counterexample the checker found and explain why it is one. Then explain why restoring the assertion allows the induction step to succeed.

What to Do When Verification Fails. There are two situations to consider, depending on whether the assertions and depth are known to be correct.

In this project the assertions and depth are provided, so the design is most likely at fault if verification fails. Each failing check produces its own counterexample waveform; open these in VaporView and use them to locate the bug. A base-case waveform is always genuine; an induction-step waveform should likewise point to the fault, but if you are convinced it is spurious, discuss it with your instructor before proceeding.

In general, when writing your own assertions, an induction failure may reflect a genuine design bug or merely a spurious counterexample. If you believe the counterexample is spurious, first try increasing the depth: a longer lead-in may be enough to rule it out. If that does not help, consider what additional `assert` statements could exclude the unreachable states the checker is exploiting.

Feasibility of Formal Verification. In principle, formal verification of finite-state designs such as digital circuits is decidable: for a correct design, there exists an inductive invariant that proves the property. In practice, however, two barriers limit its applicability. First, the search space grows exponentially with the number of state bits, so large designs may be computationally intractable. Second, finding the necessary assertion strengthening for k -induction cannot in general be automated: the auxiliary assertions needed to rule out spurious counterexamples may be non-obvious and require human insight. At the scale of this project, a complete proof is typically achievable. For industrial-scale designs, formal verification remains an active research area, and engineers sometimes accept bounded model checking as a practical substitute.

4. Stopwatch

In Assignment 1 we developed a settable watch; by the end of this assignment it will be extended with a stopwatch, a timer, and display brightness control. Because these features are loosely coupled, we begin by setting the settable watch aside and focusing purely on the stopwatch.

The board wrapper from Assignment 1 is reused here, so we need only concern ourselves with the interface defined by the `user_top` module created there.

4.1. Specification

The stopwatch displays minutes, seconds, and centiseconds on the seven-segment displays. The two buttons operate as follows.

Button	Function
<code>button[0]</code>	Start / Stop
<code>button[1]</code>	Lap (while running) / Reset (while stopped)

Start/Stop (`button[0]`) Pressing `button[0]` toggles the stopwatch between running and stopped. It does not affect the display state (live or frozen). The first centisecond increment must occur 10 ms after the button press, not sooner.

Lap and Reset (`button[1]`) The behaviour of `button[1]` depends on the current state:

- While running with a live display, pressing `button[1]` freezes the display at the current time; the stopwatch continues running in the background.
- While running with a frozen display, pressing `button[1]` unfreezes the display.
- While stopped with a frozen display, pressing `button[1]` unfreezes the display; the time is not reset.
- While stopped with a live display, pressing `button[1]` resets the time to zero; the stopwatch remains stopped and the display remains live.

Simultaneous Presses If both buttons are pressed on the same clock edge, both are ignored.

4.2. Design Approach

While this sounds straightforward, particularly given that modules from Assignment 1 can be reused, implementing the stopwatch as a single monolithic module is harder than it appears. A well-decomposed design keeps each module focused on at most one or two non-trivial decisions.

Report 5

Before reading further, sketch how you would approach the design.

Report 6

Using a feature layering approach, how would you decompose the stopwatch into layers? Describe the responsibility of each layer.

We propose to first develop a lower-level module, `stopwatch_counter`, with two control inputs: `rst`, which resets the count to zero, and `enable`, which enables the counter while high. The requirement that the first tick occurs 10 ms after the start button is pressed imposes a non-trivial timing constraint on this module, making it worthy of treatment as a unit on its own.

We should verify that the module can accommodate the lap feature without modification; if not, the design would need to be rethought.

Fortunately, it can. Freezing the display is achieved simply by taking a snapshot of the counter output and displaying that instead of the live value, so `stopwatch_counter` need not know anything about laps. This validates the design and identifies a second module, `snapshot_mux`, responsible for taking snapshots and selecting which value to display.

The resulting design has a natural structure. The *data path* (the chain of modules through which the time value flows toward the display) runs from the stopwatch counter through the snapshot multiplexer to the display driver in the board wrapper. Alongside it sits a small set of *control signals*: reset and enable for the counter, and a freeze signal for the multiplexer. A separate module will drive these via a simple FSM that responds to button presses.

Report 7

Draw a block diagram of the design described above, clearly showing the data path and control signals. Compare it with your initial sketch from Section 4.2 and comment on any differences.

Because `stopwatch_counter` has a non-trivial timing constraint, we keep the implementation concise by factoring out a cascade counter as a separate module. It will be driven by the `restartable_rate_generator` from Assignment 1.

4.3. Cascade Counter

```
1 module cascade_counter #(
2     parameter int N2 = 3,
3     parameter int N1 = 4,
4     parameter int N0 = 5,
5
6     // Output port widths
```



```

7     parameter int W2 = 2,
8     parameter int W1 = 2,
9     parameter int W0 = 3
10 ) (
11     input logic clk,
12     input logic rst,
13     input logic enable,
14     output logic [W2-1:0] count2,
15     output logic [W1-1:0] count1,
16     output logic [W0-1:0] count0
17 );

```

All outputs are initialised to 0.

When `rst` is high, all outputs are set to 0 on the next rising clock edge, regardless of `enable`.

When `rst` is low and `enable` is low, all outputs are unchanged.

When `rst` is low and `enable` is high, `count0` increments on each rising clock edge, wrapping from `N0-1` to 0. When `count0` wraps, `count1` increments, wrapping from `N1-1` to 0. When `count1` wraps, `count2` increments, wrapping from `N2-1` to 0.

Dependencies Only the following module is to be directly instantiated.

- `mod_n_counter`

Task Implement this module in `rtl/cascade_counter.sv`, following all conventions from Assignment 1, including elimination of all Verilator linting warnings. Then run the following command to verify it formally:

```
sby -f sby/cascade_counter.sby
```

4.4. Stopwatch Counter

```

1 module stopwatch_counter #(
2     parameter int CYCLES_PER_SECOND = 50_000_000
3 ) (
4     input logic clk,
5     input logic rst, // Takes priority over enable
6     input logic enable,
7     output logic [6:0] minutes,
8     output logic [5:0] seconds,
9     output logic [6:0] centiseconds // hundredths of a second
10 );

```

All outputs are initialised to 0.

When `rst` is high, all outputs are set to 0 on the next rising clock edge, regardless of `enable`.

When `rst` is low and `enable` is low, all outputs are unchanged.

When `rst` is low and `enable` is high, `centiseconds` increments once per centisecond, wrapping from 99 to 0. When `centiseconds` wraps, `seconds` increments, wrapping from 59 to 0. When `seconds` wraps, `minutes` increments, wrapping from 99 to 0. The first increment must occur one centisecond after `enable` goes high, not sooner.

Dependencies Only the following modules are to be directly instantiated.

- `cascade_counter`
- `restartable_rate_generator`

Task Implement this module in `rtl/stopwatch_counter.sv`, following all conventions from Assignment 1, including elimination of all Verilator linting warnings. Then run the following command to verify it formally:

```
sby -f sby/stopwatch_counter.sby
```

Report 8

Explain how your implementation satisfies the first-tick timing constraint: the first centisecond increment must occur one centisecond after `enable` goes high, not sooner.

4.5. Snapshot Multiplexer

```
1 module snapshot_mux #(
2     parameter int WIDTH = 1
3 ) (
4     input logic clk,
5     input logic hold,
6     input logic [WIDTH-1:0] d,
7     output logic [WIDTH-1:0] q
8 );
```

When `hold` is low, `q` equals `d` combinatorially — there is no clock edge involved; any change in `d` propagates to `q` immediately.

When `hold` is high, `q` is frozen at the value `d` held on the last rising clock edge before `hold` went high.

Task Implement this module in `rtl/snapshot_mux.sv`, following all conventions from Assignment 1, including elimination of all Verilator linting warnings. Then run the following command to verify it formally:

```
sby -f sby/snapshot_mux.sby
```

Report 9

Draw a diagram showing how this is implemented in hardware.

Note A more economical implementation would use a flip-flop with clock enable directly, avoiding the multiplexer entirely. The design above is presented to illustrate the range of options.

4.6. Stopwatch Control

```
1 module stopwatch_control (  
2     input logic clk,  
3     input logic rise_start_stop,  
4     input logic rise_lap,  
5     output logic counter_rst,  
6     output logic counter_enable,  
7     output logic lap_hold  
8 );
```

It is assumed that the instantiating module will use rising-edge detectors, so the two inputs are single-cycle pulses, each high for exactly one clock cycle when the corresponding button is pressed.

The module implements the FSM described implicitly in Section 4.1. The following additional constraints apply.

All outputs are initialised to 0.

The `counter_rst` output is high for exactly one clock cycle when asserted: as soon as `rise_lap` falls, the FSM exits the Reset state.

Report 10

Draw the FSM state diagram before implementing it. After completing the implementation, note any differences between your diagram and the final design, and reflect on what caused them.

Task Implement this module in `rtl/stopwatch_control.sv`, following all conventions from Assignment 1, including elimination of all Verilator linting warnings. Then run the following command to verify it formally:

```
sby -f sby/stopwatch_control.sby
```

The formal verification tool relies on `yosys` to compile the SystemVerilog code. However, `yosys` can promote `logic x=y;` into `assign x=y;`, and so may not detect that the former is an initialisation rather than a continuous assignment. An automated test bench has therefore also been provided: run `pytest`.

Hints

- Using `{counter_rst, counter_enable, lap_hold}` as the state variable makes the code easy to read and to write.
- Define the next-state logic for `counter_rst`, `counter_enable`, and `lap_hold` in three separate blocks, one per signal: two require only a single `assign` statement each, and the third requires an `always_comb` block.

Additional Oral Preparation 1

The hint above suggests encoding the FSM state directly as its output signals rather than using a separate state variable. Be prepared to explain why this works and what advantage it offers.

4.7. Stopwatch Integration

Task Duplicate `rtl/user_top.sv`, rename the file to `rtl/user_top_stopwatch_v1.sv`, and change the module name to `user_top_stopwatch_v1`. Then implement the stopwatch in accordance with Section 4.1, following all conventions from Assignment 1, including elimination of all Verilator linting warnings. Then run `pytest` to test it.

Dependencies Only the following modules are to be directly instantiated.

- `rising_edge_detector`
- `snapshot_mux`
- `stopwatch_control`
- `stopwatch_counter`

Note Formal verification is not used for `user_top_stopwatch_v1`. Integration modules are less amenable to it for two reasons. First, writing a complete formal specification at this level is as complex as writing the implementation itself. Second, what we want to check at the top level is that long sequences of inputs — press start, wait, press lap, press stop, check the display — produce the correct outputs; formal verification is not naturally suited to this, excelling instead at proving invariants and simple cause-and-effect properties of the form “event A implies event B on the next clock edge,” which is why it works well for individual submodules. Simulation, by contrast, can directly execute arbitrary input sequences, making it the right tool for integration testing.

Additional Oral Preparation 2

Be prepared to explain why formal verification is well suited to individual submodules but less suited to integration-level testing, and why simulation is preferred at the integration level.

4.8. Programming the Board

Open the Quartus project from Assignment 1, copy your `rtl/*.sv` files into the project directory (overwriting the existing ones), then go to `Project >> Add/Remove Files` and add any new `*.sv` files.

In `top_de1_soc.sv`, replace `user_top_watch_v4` with `user_top_stopwatch_v1` and save. Compile and program the board.

5. Timer

With the stopwatch complete, we now design and implement a timer.

5.1. Specification

The timer uses the same time-setting interface as the settable watch from Assignment 1. The timer has three modes: set, stopped, and running.

Stopped Mode. The display shows the current timer value. Pressing `button[0]` starts the timer if the count is greater than zero; otherwise it has no effect. The first decrement occurs not sooner than one second after the button is pressed. Holding `button[3]` for at least one second enters set mode.

Set Mode. The seconds digits flash at 2 Hz with an 80% duty cycle, indicating the currently selected field. The flashing digits are incremented by `button[1]` and decremented by `button[0]`; holding either button for more than half a second repeats at 10 Hz. A second press of `button[3]` moves to the minutes digits, a third to the hours digits, and a fourth exits set mode and returns to stopped mode with the newly set time.

Running Mode. The display counts down once per second. Pressing `button[0]` pauses the timer. When the count reaches zero, the timer stops automatically and the display holds at zero. Set mode cannot be entered while the timer is running.

5.2. Design Approach

The time-setting procedure is identical to that in Assignment 1, so the same modules can be instantiated. In fact, apart from three small differences, the timer is equivalent to a settable watch: time ticks down rather than up; time stops at zero instead of wrapping; and time can be paused and resumed.

We consider each difference in turn.

Time Ticks Down. The underlying counters simply need their direction reversed.

Time Stops at Zero. A combinational check determines whether the current count is all zeros; this produces a signal that qualifies the counter enable, preventing any further decrement once zero is reached.

Pausing and Resuming. The tick signal that drives the counters is qualified by a running/stopped flag. When the timer is paused, the tick is suppressed and the counters hold their value.

Since the required changes are small, the overall design closely mirrors that of the settable watch from Assignment 1. Try to derive it independently; if you get stuck, the settable watch implementation is there as a reference.

5.3. Editable Countdown with Borrow Out

Report 11

Find the datasheet for the 74LS193 synchronous 4-bit up/down counter. Using the schematic diagram in the datasheet, explain how the borrow output is generated and why it behaves as it does.

```
1 module editable_countdown #(
2     parameter int MAX    = 59,
3     parameter int WIDTH = 6
4 ) (
5     input  logic clk,
6     input  logic clr,
7     input  logic tick,
8     input  logic edit_mode,
9     input  logic inc,
10    input  logic dec,
11    output logic [WIDTH-1:0] count,
12    output logic borrow_out
13 );
```

This module behaves like `editable_counter` from Assignment 1, with one change and two additions: `count` decrements on each `tick` pulse (wrapping from 0 to `MAX`); `clr` sets `count` to 0 on the next rising clock edge, taking priority over all other inputs; and `borrow_out` is a new combinational output mirroring the borrow output of the 74LS193, except that it is also low when `edit_mode` or `clr` is high. The `count` output is initialised to 0.

Dependency Only the following module is to be directly instantiated.

- `up_down_counter_rst`

Task Implement the module in `rtl/editable_countdown.sv`, following all conventions from Assignment 1, including elimination of all Verilator linting warnings. Once you are happy with your implementation, the following command should report success:

```
sby -f sby/editable_countdown.sby
```

5.4. Timer Integration

Task Implement `user_top_timer_v1` in `rtl/user_top_timer_v1.sv` in accordance with Section 5.1, with no Verilator linting warnings. Develop and test one feature at a time using test benches you write yourself.

To facilitate formal verification, add two ports to your module:

```
1 module user_top_timer_v1 #(
2     parameter int CYCLES_PER_SECOND = 50_000_000
3 ) (
4     `ifndef FORMAL
5         output logic probe_running,
```

```
6     output logic [2:0] probe_mode_enable,
7 `endif
8     input logic clk,
```

Place the following at the end of your module, replacing the right-hand sides with your own signal names: `running` should be high whenever the timer is running, and `mode_enable` should be the output of the edit mode selector.

```
1 `ifdef FORMAL
2     assign probe_running = running;
3     assign probe_mode_enable = mode_enable;
4 `endif
5 endmodule
```

To check for linting errors with the probe ports included, run Verilator with the `FORMAL` flag defined:

```
verilator -DFORMAL -Wall --lint-only -y rtl rtl/user_top_timer_v1.sv
```

Once you are satisfied the design is complete, run `pytest` and the following command, which should both report success:

```
sby -f sby/user_top_timer_v1.sby
```

Report 12

Describe your integration plan: which features did you implement first, and why?

Report 13

The timer requires an FSM to manage its three modes. Explain your FSM design: what are the states, what are the transitions, and how did you encode the states? Justify your choices.

Dependencies Only the following modules are to be directly instantiated.

- `button_auto_repeat`
- `edit_mode_selector`
- `editable_countdown`
- `pwm_generator`
- `restartable_rate_generator`
- `rising_edge_detector`

Note Since `edit_mode_selector` has no reset pin, preventing a user from simultaneously triggering both run and edit modes is non-trivial. If run were given priority, the timer would immediately enter edit mode when next stopped. The simplest solution — and one the formal verification tool accepts — is to allow both modes to be active for a single clock cycle, after which edit mode takes priority and the run transition is cancelled.

5.5. Programming the Board

Copy your `rtl/*.sv` files into the Quartus project directory (overwriting the existing ones) and add any new files via [Project >> Add/Remove Files](#).

In `top_de1_soc.sv`, replace `user_top_stopwatch_v1` with `user_top_timer_v1` and save.

Compile and program the board.

6. Brightness Control

We wish to control the brightness of the seven-segment display across all three apps — watch, stopwatch, and timer — with the leftmost two switches, `sw[9]` and `sw[8]`, selecting the brightness level.

Modifying each app individually would be tedious and risk introducing errors. Instead, we create a wrapper module that sits between `top_de1_soc` and the app, intercepting the brightness-related signals without touching the app itself.

The wrapper reads `sw[9]` and `sw[8]` to determine the brightness level and passes them straight through to the app. It also intercepts the app's three blanking signals (one each for hours, minutes, and seconds): when the app asserts blanking, the wrapper passes it through unchanged; when blanking is off, the wrapper substitutes its own PWM blanking signal to control what fraction of the time the display is on. Provided the PWM period is short enough, the human eye perceives a steady reduced brightness rather than a flickering display.

6.1. Grey Code

If brightness were mapped to the switches in binary counting order (00, 01, 10, 11), smoothly increasing the brightness would require the user to transition from 01 to 10, which demands either perfect coordination or a momentary pass through 00 or 11.

Grey code avoids this problem. Rather than incrementing in standard binary, each successive value differs from the previous by exactly one bit: 00, 01, 11, 10. Moving between any two adjacent brightness levels therefore requires flipping only a single switch.

This leads to the following brightness specification.

<code>sw[9:8]</code>	Brightness	Duty cycle
00	Dim	12.5%
01	Low	25%
11	Medium	50%
10	Full	100%

A PWM period of 1 ms is chosen, placing the switching frequency well above the flicker-fusion threshold.

6.2. Brightness Wrapper

Copy `rtl/user_top_stopwatch_v1.sv` to `rtl/user_top_brightness_wrapper.sv`, rename the module to match, then delete everything inside the module body, leaving only the port list. This gives you a correctly interfaced but empty shell to build from.

Instantiate `user_top` from Assignment 1 inside the wrapper, connecting every signal directly to the corresponding wrapper port, except for the blanking outputs.

Task Implement `user_top_brightness_wrapper` as described above, using `mod_n_counter` to generate the PWM signal. Eliminate all Verilator linting warnings and errors. Formally verify your module by running:

```
sby -f sby/user_top_brightness_wrapper.sby
```

Since the PWM period and duty cycle are more naturally verified with a test bench, also run `pytest`.

Dependencies Only the following modules are to be directly instantiated.

- `mod_n_counter`
- `user_top`

Additional Oral Preparation 3

Be prepared to explain how PWM is used to control the brightness of a seven-segment display, and why the display appears steady rather than flickering.

Additional Oral Preparation 4

Be prepared to explain the wrapper pattern: what a wrapper module is, how it intercepts signals between two other modules without modifying either, and what advantage this offers when you want to add new behaviour to an existing design.

7. Integration

Our final design has three nested modules at the top level.

- `top_de1_soc` will instantiate `user_top_brightness_timepiece`; but make this change only inside Quartus (or you will break the automated tests).
- `user_top_brightness_timepiece` will instantiate `user_top_timepiece_v1`, providing variable brightness to our timepiece.
- `user_top_timepiece_v1` will bring together our watch, stopwatch and timer.

7.1. Timepiece

The template below introduces a concept borrowed from C: a **struct**.

A **struct** groups related signals under a single name. In SystemVerilog, **struct packed** additionally requires all fields to be stored as one contiguous sequence of bits, so the whole structure behaves like a single logic vector: a packed struct can be assigned en masse.

Two packed struct types are defined here: **ui_in_t** bundles the user-interface inputs needed by each submodule (**button** and **sw**), while **ui_out_t** bundles each submodule's outputs. This makes the multiplexer logic at the bottom of the module easier to write.

Note that Icarus Verilog does not handle arrays of packed structs reliably. The template therefore avoids such arrays: each submodule gets its own named **ui_in_t** and **ui_out_t** variable rather than an indexed element of an array.

```
1  `timescale 1ns / 1ps
2
3  module user_top_timepiece_v1 #(
4      parameter int CYCLES_PER_SECOND = 50_000_000
5  ) (
6      input logic clk,
7      input logic [3:0] button,
8      input logic [9:0] sw,
9      output logic [9:0] led,
10     output logic [6:0] hours_disp,
11     output logic [6:0] minutes_disp,
12     output logic [6:0] seconds_disp,
13     output logic blank_hours,
14     output logic blank_minutes,
15     output logic blank_seconds
16 );
17
18     typedef struct packed {
19         logic [3:0] button;
20         logic [9:0] sw;
21     } ui_in_t;
22
23     typedef struct packed {
24         logic [9:0] led;
25         logic [6:0] hours_disp;
26         logic [6:0] minutes_disp;
27         logic [6:0] seconds_disp;
28         logic blank_hours;
29         logic blank_minutes;
30         logic blank_seconds;
31     } ui_out_t;
32
33     ui_in_t watch_in, timer_in, sw_in;
34     ui_out_t watch_out, timer_out, sw_out;
35
36     // TO DO -- instantiate:
37     // user_top_watch_v4
38     // user_top_timer_v1
39     // user_top_stopwatch_v1
40     //
41     // Connect the ports up to the appropriate ui_in and ui_out
```

```

42 // e.g., for the watch, .button(watch_in.button)
43
44 // -----
45 // Multiplexers
46 // -----
47
48 ui_in_t ui_top_in;
49 assign ui_top_in.sw = sw;
50 assign ui_top_in.button = button;
51
52 ui_in_t ui_top_in_no_buttons;
53 assign ui_top_in_no_buttons.sw = sw;
54 assign ui_top_in_no_buttons.button = '0;
55
56 ui_out_t ui_top_out;
57 assign led = ui_top_out.led;
58 assign hours_disp = ui_top_out.hours_disp;
59 assign minutes_disp = ui_top_out.minutes_disp;
60 assign seconds_disp = ui_top_out.seconds_disp;
61 assign blank_hours = ui_top_out.blank_hours;
62 assign blank_minutes = ui_top_out.blank_minutes;
63 assign blank_seconds = ui_top_out.blank_seconds;
64
65 logic [1:0] mode_sel;
66 assign mode_sel = sw[1:0];
67 always_comb
68     case (mode_sel)
69         // Stopwatch
70         2'b01: begin
71             // TO DO -- fill in appropriately
72         end
73         // Timer
74         2'b11: begin
75             // TO DO -- fill in appropriately
76         end
77         // Watch
78         default: begin
79             // TO DO -- fill in appropriately
80         end
81     endcase
82 endmodule

```

Task Complete the template above so that the user top-level module runs a watch, stopwatch, and timer simultaneously. The user selects the active app based on the positions of switches 1 and 0. The buttons are sent only to the currently selected app, whereas all switches are sent to all apps simultaneously. Run `pytest`.

Additional Oral Preparation 5

Be sure to understand how packed structs simplify routing large numbers of connections through multiplexers.

7.2. Brightness Wrapper

Tasks

- Copy the module `user_top_brightness_wrapper`.
- Rename the copy to `user_top_brightness_timepiece`.
- Inside the copied module, replace `user_top` with `user_top_timepiece_v1`.
- Run `sby -f sby/user_top_brightness_wrapper.sby`.
- Run `pytest tests/test_user_top_brightness_wrapper.py`.

Note It is reasonable to ask whether `user_top_brightness_timepiece` should have its own tests. In theory, because both `user_top_brightness_wrapper` and `user_top_timepiece_v1` are tested, additional tests may be unnecessary. On the other hand, the code replication and minor edits were not automated; even if they were, we would still need to check that the process produced the correct result.

7.3. Programming the Board

Open the Quartus project from Assignment 1, copy your `rtl/*.sv` files into the project directory (overwriting the existing files), then go to **Project** > **Add/Remove Files** and add any new `*.sv` files.

In `top_de1_soc.sv`, replace `user_top_timer_v1` with `user_top_brightness_timepiece` and save.

Compile and program the board.

The table below summarises the switch mappings used in the integrated design: brightness is controlled by `sw[9:8]`, and app selection is controlled by `sw[1:0]`.

<code>sw[9:8]</code>	Brightness	Duty cycle	<code>sw[1:0]</code>	Mode
00	Dim	12.5%	00	Watch
01	Low	25%	01	Stopwatch
11	Medium	50%	11	Timer
10	Full	100%	10	Watch

Report 14

Looking back across both assignments, reflect on your experience designing and implementing a digital watch on an FPGA. What went well, what was harder than expected, and what would you do differently if you were starting over?

Report 15

Estimate how much time you saved compared to having used only Quartus. Break down the time savings between better editing features, linting, and automated tests plus formal verification. Without these tools, at what stage would you have discovered a problem, and how much harder would it have been to locate and fix the source of the error?

8. Synthesis

The environment includes Yosys, a synthesis tool developed at TU Wien. Its open, inspectable architecture lets you observe in detail how a synthesiser translates Verilog into hardware.

Verilog serves dual purposes: simulation and synthesis. This exercise explores the fundamental difference between them. Verilog originated as a *simulation* language — simulators run your code to predict circuit behaviour. Synthesis is fundamentally different: a synthesis tool must translate that intended behaviour into physical hardware. You will step through the Yosys synthesis stages below, examining what happens at each step.

Open a terminal and run `yosys`.



```
(venv) root@d16985fc7d9d:/workspace# yosys

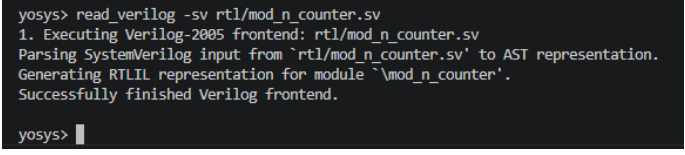
/-----\
| yosys -- Yosys Open SYnthesis Suite                               |
|                                                                     |
| Copyright (C) 2012 - 2020 Claire Xenia Wolf <claire@yosyshq.com>   |
|                                                                     |
| Permission to use, copy, modify, and/or distribute this software for any |
| purpose with or without fee is hereby granted, provided that the above |
| copyright notice and this permission notice appear in all copies.     |
|                                                                     |
| THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES |
| WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF      |
| MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR |
| ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES |
| WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN |
| ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF |
| OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.     |
|-----\

Yosys 0.33 (git sha1 2584903a060)

yosys> |
```

At the `yosys>` prompt, read `rtl/mod_n_counter.sv`:

```
read_verilog -sv rtl/mod_n_counter.sv
```



```
yosys> read_verilog -sv rtl/mod_n_counter.sv
1. Executing Verilog-2005 frontend: rtl/mod_n_counter.sv
Parsing SystemVerilog input from `rtl/mod_n_counter.sv' to AST representation.
Generating RTLIL representation for module `mod_n_counter'.
Successfully finished Verilog frontend.

yosys> |
```

Next, set the top-level module and produce a diagram of the elaborated design:

```
hierarchy -top mod_n_counter
show -format png -prefix elaborated
```

Passing `-format png` and `-prefix` directs `show` to save a `.png` file rather than launch a graphical viewer. The file is written to the current working directory (`/workspace`); click on it in the VS Code Explorer panel to open it.

Report 16

Include `elaborated.png` in your report. Yosys has elaborated the design but has not yet synthesised anything. The diagram contains five numbered elements (\$1 to \$5). Two are labelled PROC: one cites line numbers from `rtl/mod_n_counter.sv` and corresponds to the `always_ff` block; the other, at line 0.0–0.0, represents the initialisation of `count`. The remaining three cells are \$1t, \$add, and \$mux. What does each compute? Explain how the three work together to implement the counter’s next-state logic.

Run `proc` to convert procedural blocks into logic, then `opt` to simplify, and produce a new diagram:

```
proc
opt -full
show -format png -prefix proc
```

Report 17

Include `proc.png` in your report. The two PROC cells from `elaborated.png` have been replaced by a single cell labelled \$sdffe. What do you think \$sdffe stands for? It has four ports: CLK, D, EN, and SRST; what does each correspond to in the original `always_ff` block? The combinational cells \$1t, \$add, and \$mux are unchanged. What does this tell you about what `proc` and `opt` did and did not change?

Run `techmap` to replace Yosys internal cells with technology-independent gate primitives, then optimise:

```
techmap
opt -full
show -format png -prefix techmap
```

Report 18

Include `techmap.png` in your report. The cell names now carry underscores and suffixes: \$sdffe has become \$_SDFFE_PPOP_ and the combinational cells have become \$_XOR_, \$_NOT_, \$_AND_, and \$_MUX_. These underscore-prefixed names are Yosys’s technology-independent gate primitives, one step closer to real hardware. Notice that there are now two flip-flop instances rather than one; why? Observe also the labelled bubbles on the wires: 1:0 denotes a 2-bit wire (bits 1 and 0), 0:0 a single-bit wire (bit 0 only), and 1:1 -- 0:0 means bit 1 of the source connects to bit 0 of the destination. Why does Yosys slice the wires in this way? Finally, the suffix PPOP encodes the polarities of the flip-flop’s ports.^a Can you work out what each letter represents?

^aAt the time of writing, \$_SDFFE_PPOP_ is documented in the [Yosys manual](#).

Finally, run `abc` to map the logic to AND, OR, XOR, NAND, and NOR gates, then optimise and produce a final diagram:

```
abc -g AND,OR,XOR,NAND,NOR
opt -full
show -format png -prefix abc
```

Note that `abc` always treats NOT as a primitive, even when it is not listed in the `-g` argument.

Report 19

Include `abc.png` in your report. Notice that the `$_MUX_` present in every previous diagram has disappeared: `abc` replaced it with pure logic gates. The combinational logic has been reduced to just a `$_NOT_` and a `$_XOR_`, feeding two `$_SDFFE_PPOP_` flip-flops. From the diagram, draw the corresponding circuit: show the two flip-flop outputs (the two bits of `count`), the NOT gate, the XOR gate, and how they connect. Then, using truth tables, verify that this circuit correctly implements the counter's next-state logic for all four states.

The gate-level view produced here is directly useful in two contexts: building a circuit from discrete logic chips (such as 74-series ICs), or targeting an ASIC, where the design is mapped to a library of standard cells that are physical implementations of logic gates. On an FPGA, however, the target is look-up tables (LUTs) rather than gates. A real FPGA synthesis flow would first tell Yosys the LUT size available on the target device, then instruct `abc` to map the logic to LUTs of that size.

Type `exit` to quit Yosys.

9. Enhancements

You are encouraged to add extra features to your digital watch. Some ideas are listed below.

- A calendar display showing the current date alongside the time.
- A world clock that cycles through the time in several time zones.
- An alarm that flashes the display at a set time.
- An hourly chime that plays a short LED animation on the hour.
- A power-saving sleep mode that dims the display after a period of inactivity and eventually turns it off.
- A PIN-protected mode that requires a correct code before the time can be edited.
- A secret button combination that makes your student ID scroll across the display.
- A game, such as a reaction-time tester.
- Preset intervals that instantly load a popular countdown time — for example, 1 minute, 5 minutes, or 25 minutes (a Pomodoro interval) — selectable via a button combination.
- Button-controlled brightness: cycle through brightness levels with a dedicated button, eliminating the need for a switch. With many levels available, holding the button could scroll through them slowly at first and then accelerate, mirroring the auto-repeat behaviour used when setting the time.
- Button-controlled mode selection: the standard integration uses two switches to select between watch, stopwatch, and timer modes; replace them with a button that cycles through the modes on each press.

Report 20

If you add extra features, describe what you implemented, how you implemented it, and the reasoning behind any major design decisions.

Report 21

Based on your experience across both assignments, explain which parts of a digital watch are well suited to an FPGA and which would be easier to implement in software. Assume that any component responsible for counting or tracking time requires clock-cycle accuracy.

A. Common Pitfalls

See Assignment 1 for common pitfalls that apply across the whole project.

B. Formal Verification Environment

Formal verification relies on three tools working together as a pipeline: Yosys, Z3, and SymbiYosys (`sby`).

Yosys Yosys (Yosys Open Synthesis Suite) is an open-source framework for synthesising hardware designs, first released in 2012. For formal verification, Yosys reads your SystemVerilog source and compiles it into a mathematical model of the circuit in SMT-LIB format, a standard language for posing logical problems to automated reasoning tools.

Z3 Z3 is an SMT (Satisfiability Modulo Theories) solver developed by Microsoft Research and open-sourced in 2015. Given a set of logical constraints, it either finds an assignment of values that satisfies them all, or proves that no such assignment exists. In hardware verification, Z3 receives the circuit model from Yosys and searches for any input that would violate your assertions: if it finds one, that input is a counterexample; if it finds none, the property is verified.

SymbiYosys SymbiYosys is a front-end tool that orchestrates the verification workflow. Given your SystemVerilog source and a small configuration file, it invokes Yosys to produce the circuit model, passes it to Z3, and reports whether each property passes or fails. It is, in short, the glue between the other two tools: Yosys handles the hardware, Z3 handles the reasoning, and `sby` drives the process.

A note on SystemVerilog Assertions The IEEE SystemVerilog standard includes a sub-language called SVA (SystemVerilog Assertions), which provides a concise, declarative syntax for specifying properties a design must satisfy. Ideally we would use SVA, but full SVA support in open-source Yosys requires the Verific frontend, a commercial tool that must be licensed separately, and no open-source alternative currently exists. We therefore write assertions in ordinary SystemVerilog, which is more verbose but expresses the same goal: properties the solver must prove hold for all possible inputs.

C. Version History

Version	Date	Changes
1.2	2026-09-05	Stylistic changes.
1.1	2026-05-11	Added test bench to catch errors missed by <code>yosys</code> .
1.0	2026-05-02	Initial release.

C.1. Changes in Version 1.2

- Stylistic changes.

C.2. Changes in Version 1.1

- Added an automated cocotb test bench for `stopwatch_control` to catch uses of initialisation syntax (`logic x=y;`) that `yosys` may silently promote to continuous assignment (`assign x=y;`) and therefore not flag as an error.